IU2541230002 - CPP01 - PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

C++ Console-Based Object-Oriented Programming Project

Enrollment No.:	IU2541230002
Student Name:	_____
Semester:	3
Subject:	C++ Guided Project
Date of Submission:	13-10-2026

CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. System / Project Flow
6. Class Descriptions
7. OOP Concepts Used
8. UML Diagrams
9. Implementation Details
10. Testing
11. Screenshots
12. GitHub Repository
13. Conclusion and Future Scope

1. Introduction

A library has to keep track of a large number of books, the people who are registered to borrow them, and the daily issue and return of those books. When this work is done on paper it is slow, and mistakes such as lost records, duplicate entries and wrongly calculated fines are common.

The **Library Management System** is a console-based application written in C++ using Object-Oriented Programming. It lets a librarian add, view, search, update and delete books and members, issue and return books, calculate fines for late returns, and produce a summary report. All data is stored in text files so that nothing is lost when the program is closed.

The program is organised around six main classes (Book, Member, Person, Transaction, Library and FileManager). The Library class acts as the system class: *main()* only shows the menu and calls its functions, while the entity classes hold the data and the FileManager class saves and loads it.

2. Problem Statement

Manual library record keeping has several problems: books and members may be recorded twice, it is hard to know quickly which member holds which book, a book can be given to two people by mistake, due dates and fines are calculated by hand, and records can be damaged or lost.

The problem is to design and implement a C++ program that stores book, member and transaction records in an organised way, allows a book to be issued to only one member at a time, limits each member to 3 books, calculates a fine after the 14-day borrowing period, checks all input for errors, and saves the data permanently in files.

3. Objectives

- To design the system using classes and objects with proper encapsulation (Book, Member, Person, Transaction, Library, FileManager).
- To use default and parameterized constructors and destructors correctly.
- To demonstrate inheritance (Member derived from Person) and runtime polymorphism using a virtual display() function.
- To use dynamic memory (new[] and delete[]) for storing a growing number of records.
- To implement the main process of issuing and returning books with automatic fine calculation.
- To store and retrieve data permanently using file handling.
- To validate every user input and handle errors with exception handling so that the program does not crash.
- To provide a clear menu-driven interface and document the design with UML diagrams.

4. Functional Requirements

The system provides the following functions, which match the nine options of the main menu:

ID	Requirement
FR1	Add a book (unique ID, title, author, category) and add a member (unique ID, name, 10-digit phone, e-mail).
FR2	Display all books, all members or all transactions in a formatted table.
FR3	Search books by ID or by keyword (title, author, category) and members by ID or name.
FR4	Update a book or member; pressing Enter keeps the old value of a field.
FR5	Delete a book or member, but not a book that is currently issued or a member who still holds books; ask for confirmation first.
FR6	Issue a book: member and book must exist, the book must be available, and the member must hold fewer than 3 books.
FR7	Return a book: close the transaction, make the book available again and calculate the fine (Rs. 5 per day after 14 days).
FR8	Generate a report: total books, available/issued books, members, transactions, fines collected and outstanding, books by category and the list of currently issued books.
FR9	Save all data to files and load it back; data is loaded automatically at start-up and the user is asked to save on exit.
FR10	Validate all input (numbers, phone, e-mail, dates) and show a clear message for every error.

Non-functional requirements: the program is easy to use (numbered menus), reliable (invalid input never crashes it), portable (standard C++11, no operating-system specific code) and maintainable (one class per file with clear responsibilities).

5. System / Project Flow

The flow of the project from user input to file storage is:

USER -> MAIN MENU (main.cpp) -> SYSTEM CLASS (Library) -> ENTITY CLASSES (Book, Member, Transaction) -> MAIN TRANSACTION / PROCESS -> FILE MANAGER -> DATA FILES

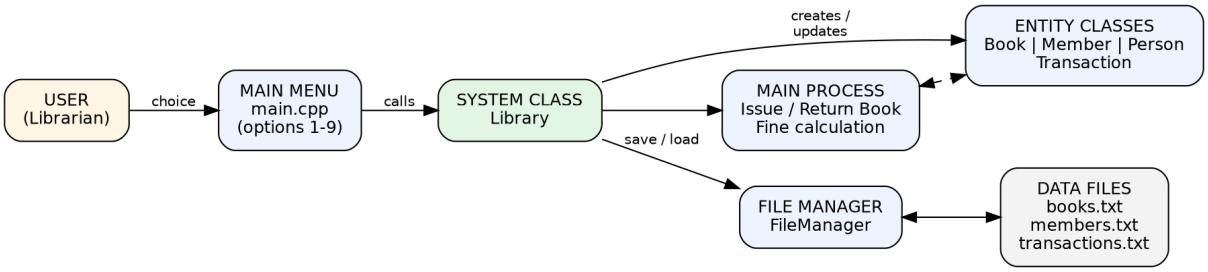


Figure 1: Project flow

Step-by-step flow for issuing a book:

- The user selects option 6 in the main menu; main() calls Library::transactionMenu().
- Library::issueBook() asks for the Member ID, Book ID and issue date, and checks that the member and book exist.
- It checks that the book is available and that the member can still borrow (fewer than 3 books). All checks are made before any data is changed.
- A Transaction object is created, Book::issue() marks the book as issued and Member::borrowBook() increases the borrowed count.
- When the user saves (option 8 or on exit), Library::saveAll() calls the FileManager, which writes books.txt, members.txt and transactions.txt.

Step-by-step flow for returning a book:

- Library::returnBook() asks for the Book ID, finds the open transaction for that book and asks for the return date.
- Transaction::closeTransaction() checks that the return date is not before the issue date and stores it.
- Book::returnBook() and Member::returnBook() update the status and the borrowed count.
- Transaction::daysLate() and calculateFine() give the fine, which is displayed to the user and later saved to the file.

6. Class Descriptions

Class	Data Members	Member Functions	Responsibility
Person	id, name, phone (protected)	Person(), Person(id, name, phone), virtual ~Person(), virtual display(), getId(), getName(), getPhone(), setName(), setPhone()	Base class that stores the common details of a person and defines the virtual display() function that derived classes override.
Member	email, borrowedCount, MAX_BORROW (static = 3); inherits id, name, phone	Member(), Member(id, name, phone, email, count), display() [overrides Person], borrowBook(), returnBook(), canBorrow(), getMemberId(), getEmail(), setEmail()	A library member derived from Person. Tracks how many books the member holds and enforces the limit of 3 books.
Book	bookId, title, author, category, available	Book(), Book(id, title, author, category, available), display(), issue(), returnBook(), isAvailable(), getters, setTitle(), setAuthor(), setCategory()	Represents one book and its availability. issue() and returnBook() change the status and reject invalid changes.
Transaction	transactionId, bookId, memberId, issueDate, returnDate; LOAN_DAYS = 14, FINE_PER_DAY = 5 (static)	Transaction(), Transaction(id, bookId, memberId, issue, return), display(), closeTransaction(), isReturned(), daysLate(), calculateFine()	Records one issue/return event, validates dates and calculates the late-return fine.
Library	Book* books, Member* members, Transaction* transactions, counts and capacities, nextTransactionId, FileManager fileMgr	Library(), ~Library(), addRecord(), displayRecords(), searchRecord(), updateRecord(), deleteRecord(), transactionMenu(), report(), saveLoadMenu(), searchBook(), issueBook(), returnBook(), saveAll(), loadAll()	System class. Owns the dynamic arrays, applies all business rules, and connects the entity classes to the FileManager.
FileManager	booksFile, membersFile, transactionsFile (file names)	FileManager(), saveBooks(), loadBooks(), saveMembers(), loadMembers(), saveTransactions(), loadTransactions()	Performs all file input/output. Converts objects to text lines and back, and skips damaged lines while loading.

Supporting classes: the Utils namespace provides validated console input (readInt, readText, readPhone, readEmail, readDate, confirm) and date helpers (isValidDate, dateToDays, today). LibraryException and InputEndedException, derived from std::runtime_error, are used to report every error in the program.

7. OOP Concepts Used

Concept	Where Used	Explanation
Classes and Objects	Book, Member, Person, Transaction, Library, FileManager	Every real-world entity is a class; objects such as Book b(101, ...) hold the actual data.
Encapsulation	Data members of every class	All data members are private or protected. They are read through getters and changed only through setters such as setPhone(), which reject invalid values.
Constructor	Book(), Book(...), Member(...), Transaction(...), Library()	Each entity has a default constructor (needed by new[]) and a parameterized constructor that validates its arguments and throws on bad data.
Destructor	Library::~Library(), Person::~Person()	Library's destructor releases the three dynamic arrays with delete[]. Person's destructor is virtual so deletion through a Person pointer is safe.
Inheritance	class Member : public Person	Member reuses id, name and phone from Person and adds email and borrowedCount.
Runtime Polymorphism	Person::display() (virtual), Member::display(), Library::displayRecords(), printPerson()	A Person* (or Person&) that refers to a Member calls Member::display() at run time. The base printCore() is reused by the override.
Dynamic Memory	Library constructor and growArray(); FileManager load functions	Arrays such as new Book[capacity] are created at run time because the number of records is unknown. When full, the capacity is doubled (new[], copy, delete[]). Loading from files also allocates a new array.
File Handling	FileManager (ifstream / ofstream)	Books, members and transactions are stored in three text files, loaded automatically at start-up and saved from the menu or on exit.
Exception Handling	LibraryException thrown in all classes; try/catch in main() and FileManager	Duplicate IDs, invalid data, rule violations and file errors throw exceptions that main() catches and reports without crashing.
Validation	Utils::readInt(), readPhone(), readEmail(), readDate()	Input is checked (numbers only, 10-digit phone, e-mail format, real calendar dates) and the user is asked again until valid.

Why dynamic memory is needed: the number of books, members and transactions is not known when the program is written and changes while it runs. A fixed array would either waste memory or run out of space. With new[] the arrays start small (10 elements) and grow by doubling when they are full, and delete[] gives the memory back when the Library object is destroyed or when data is reloaded from files.

8. UML Diagrams

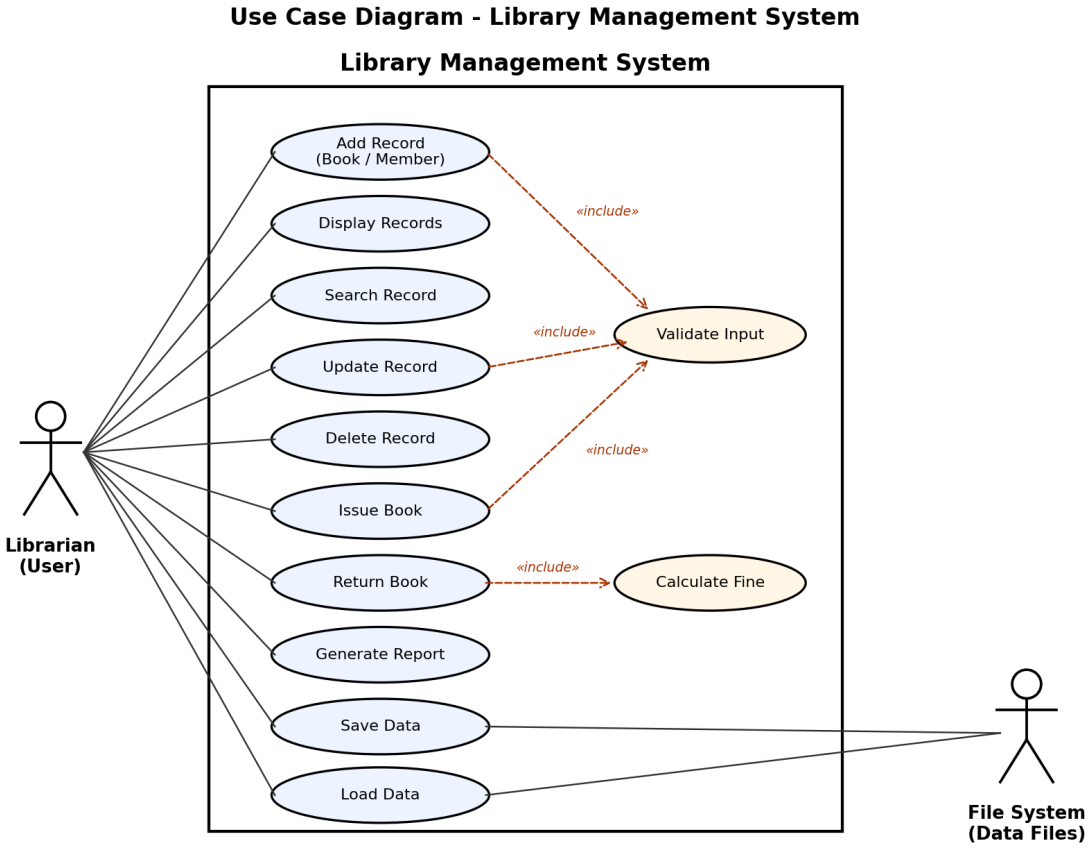


Figure 2: Use Case Diagram

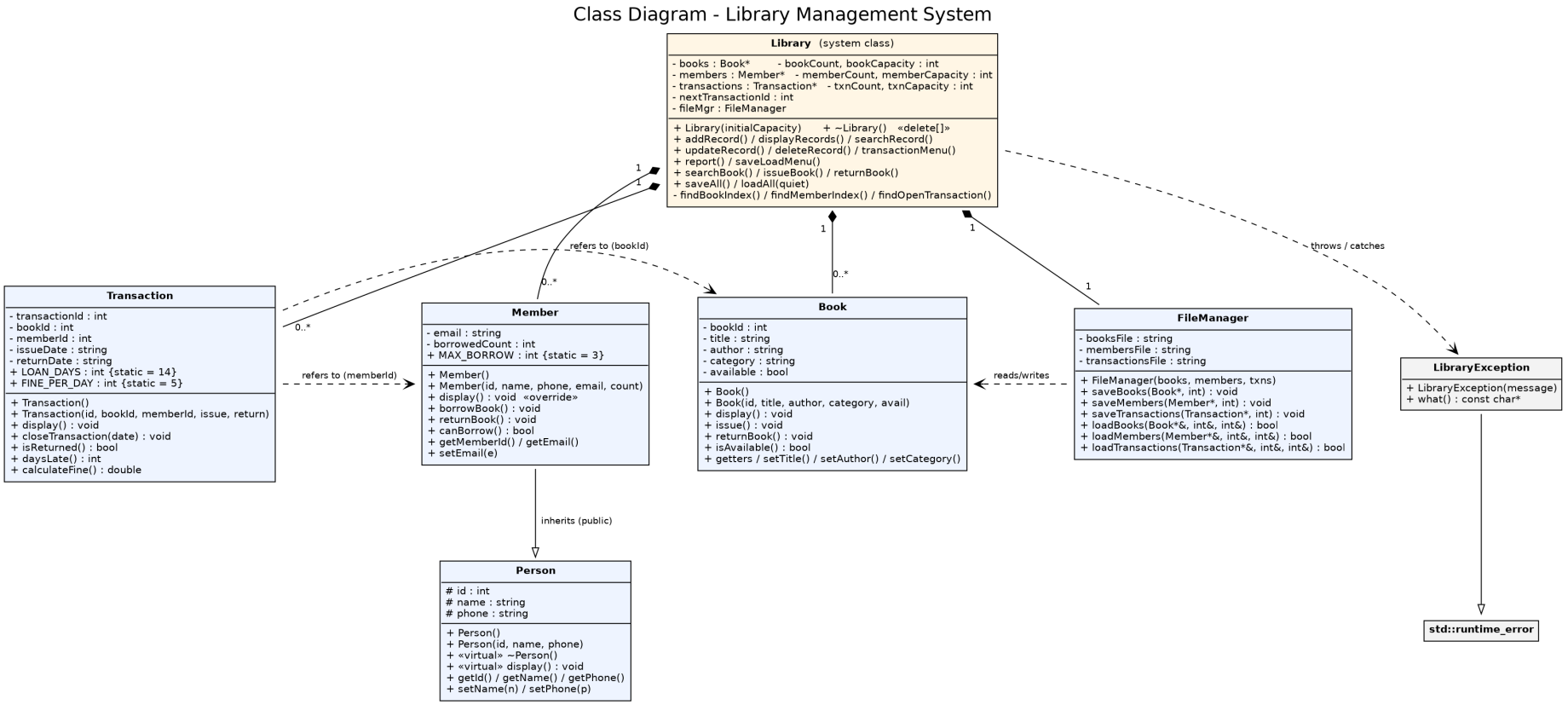


Figure 3: Class Diagram

The class diagram shows that Member inherits from Person, that Library owns arrays of Book, Member and Transaction objects together with one FileManager (composition), and that Transaction refers to a book and a member by their IDs.

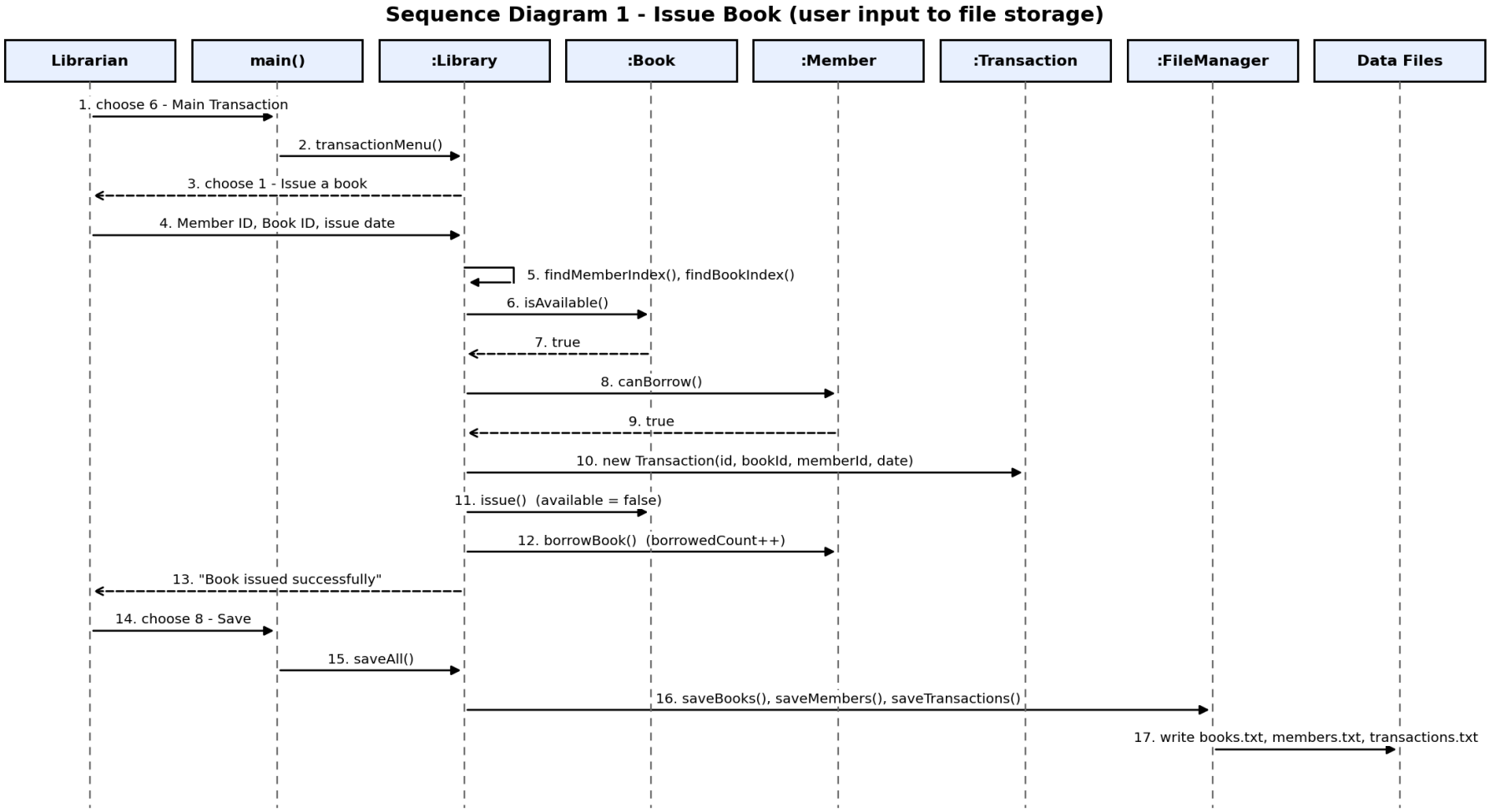


Figure 4: Sequence Diagram 1 - Issue Book

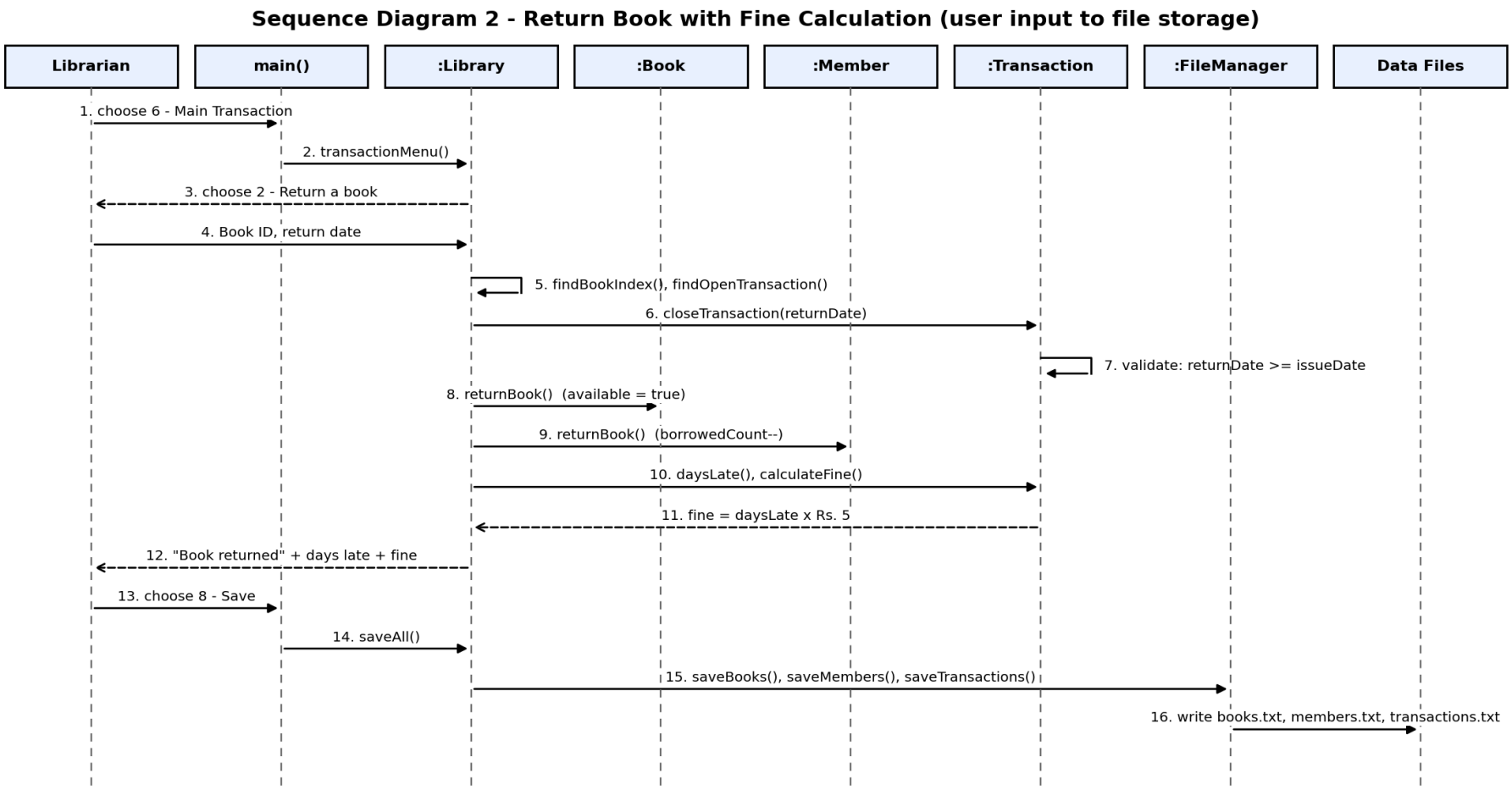


Figure 5: Sequence Diagram 2 - Return Book with fine calculation

9. Implementation Details

Files of the project: main.cpp, Library.h/.cpp, Person.h/.cpp, Member.h/.cpp, Book.h/.cpp, Transaction.h/.cpp, FileManager.h/.cpp, Utils.h/.cpp and LibraryException.h. Every class has its own header and source file.

Business rules: a member may hold at most 3 books; the free borrowing period is 14 days; the fine is Rs. 5 for every day after that; a book that is issued cannot be issued again or deleted; a member who still holds books cannot be deleted; the return date cannot be earlier than the issue date.

Data files: three text files with fields separated by the | character. books.txt: bookId|title|author|category|available. members.txt: memberId|name|phone|email|borrowedCount. transactions.txt: transactionId|bookId|memberId|issueDate|returnDate (a dash means not yet returned). The | character is not allowed in typed text so that the files stay readable.

Design decisions: (1) all rules are checked before any object is changed, so a failed operation never leaves half-updated data; (2) the Library class cannot be copied, which prevents two objects from deleting the same array; (3) fines are calculated from dates, not stored, so they are always correct; (4) a damaged line in a data file is skipped with a warning instead of stopping the program; (5) the program was also compiled with AddressSanitizer and UndefinedBehaviorSanitizer and run through all test cases with no memory errors or leaks reported.

10. Testing

Fifteen test cases were executed on the compiled program. All of them passed. The full table with inputs, expected and actual outputs is given in the Submission Template; a summary is shown below.

Test ID	Operation	Result
T01	Add Record - Book	Pass
T02	Add Record - Member	Pass
T03	Duplicate ID check	Pass
T04	Phone and e-mail validation	Pass
T05	Display Records - Books	Pass
T06	Search Record - keyword	Pass
T07	Update Record - Book	Pass
T08	Issue Book	Pass
T09	Issue an already issued book	Pass
T10	Borrow limit	Pass
T11	Return Book with late fine	Pass

Test ID	Operation	Result
T12	Delete an issued book	Pass
T13	Report	Pass
T14	Save / Load persistence	Pass
T15	Invalid menu input	Pass

11. Screenshots

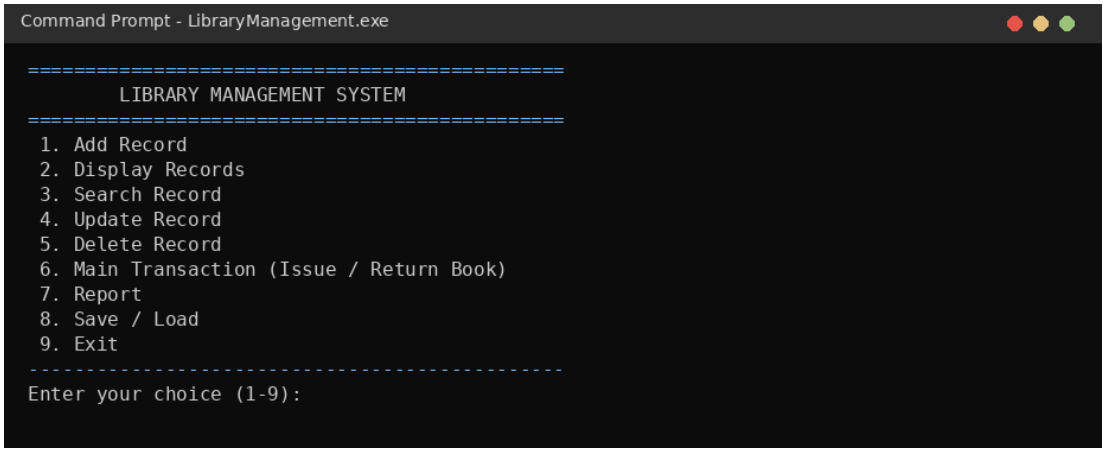


Figure 6: Main menu

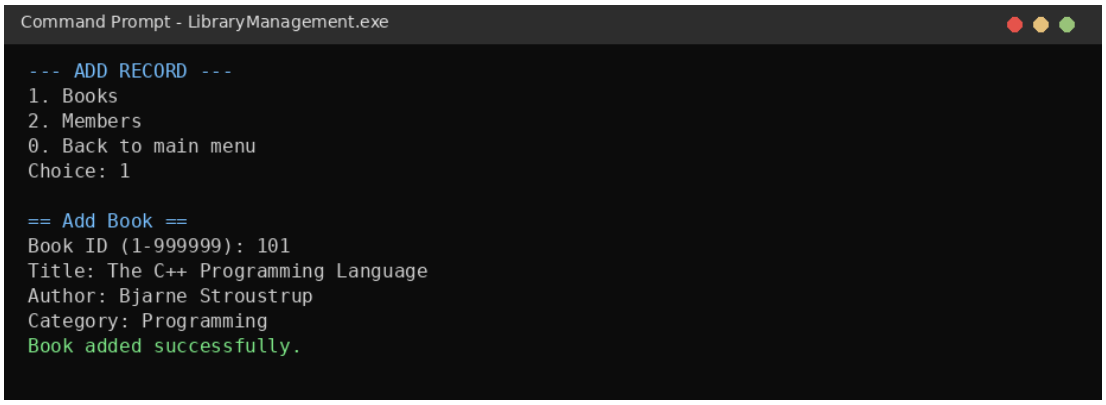


Figure 7: Adding a book

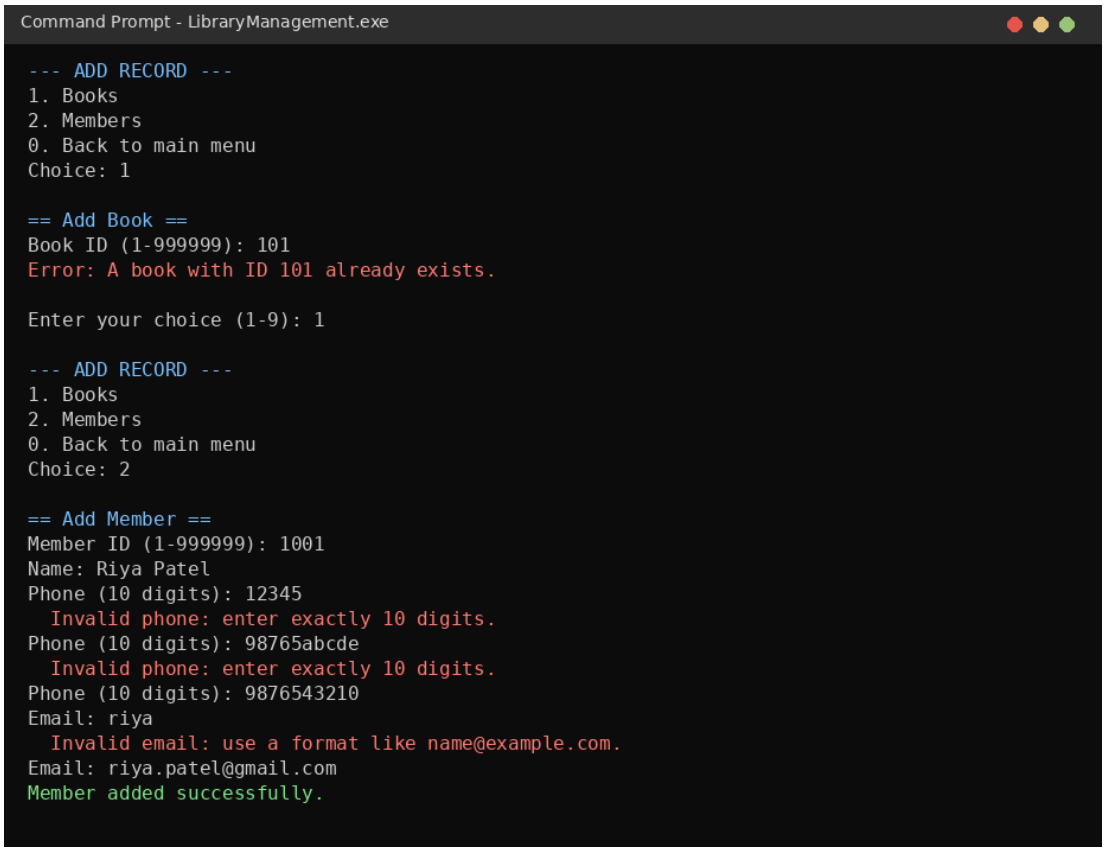


Figure 8: Duplicate ID error, phone and e-mail validation, adding a member

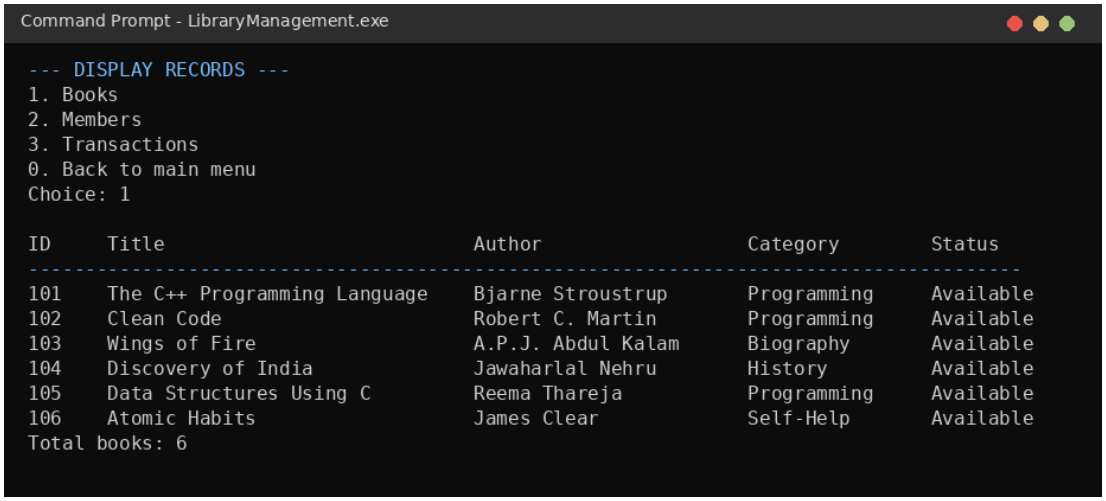


Figure 9: Displaying all books

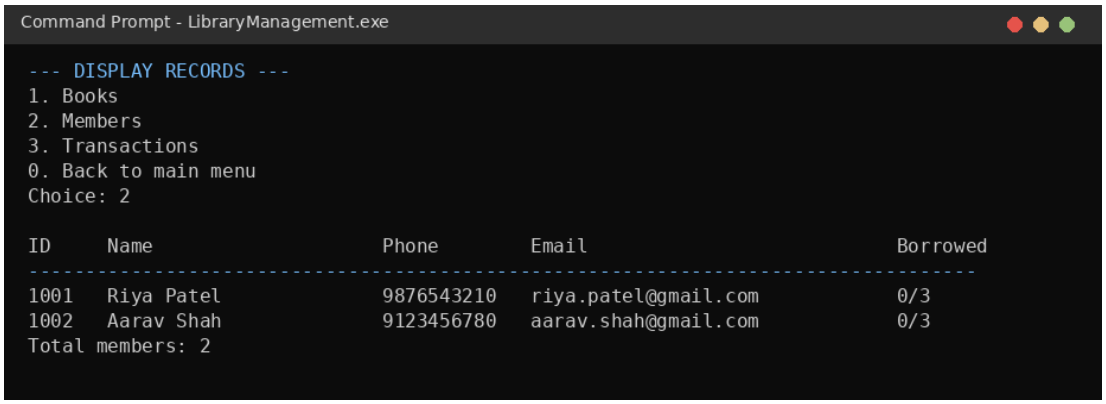


Figure 10: Displaying members (polymorphic display)

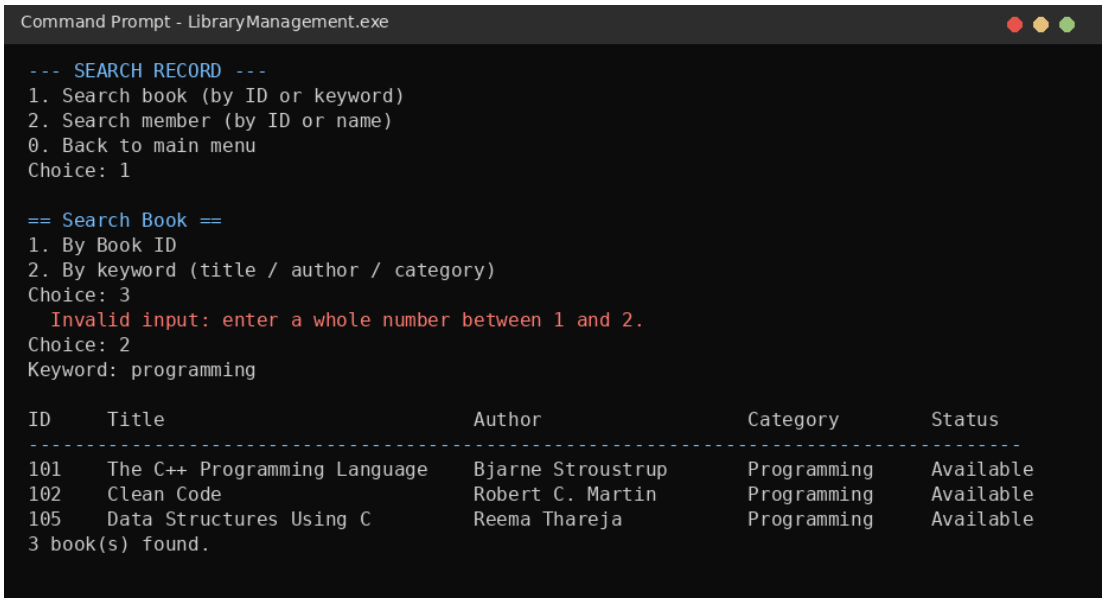


Figure 11: Searching books by keyword

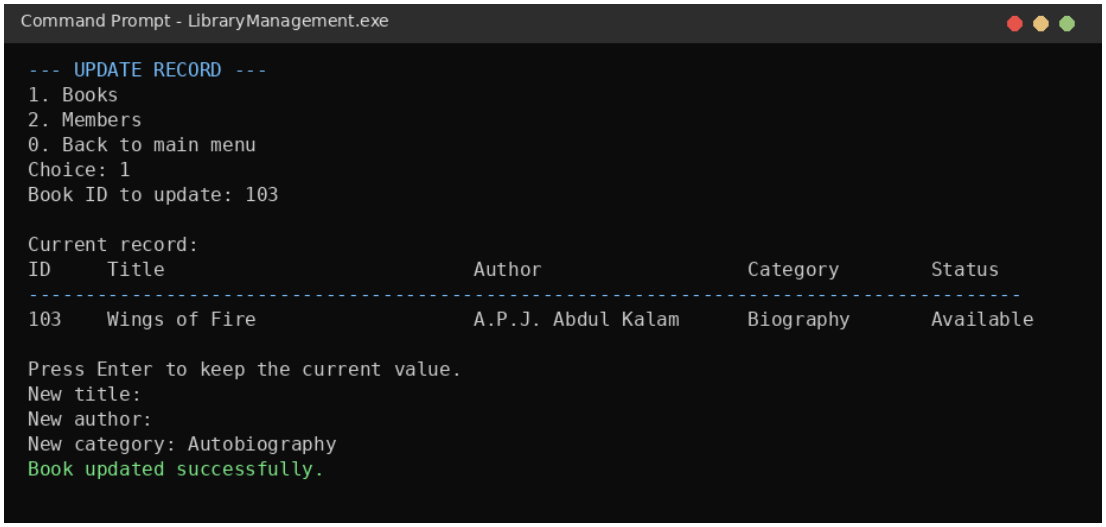


Figure 12: Updating a book

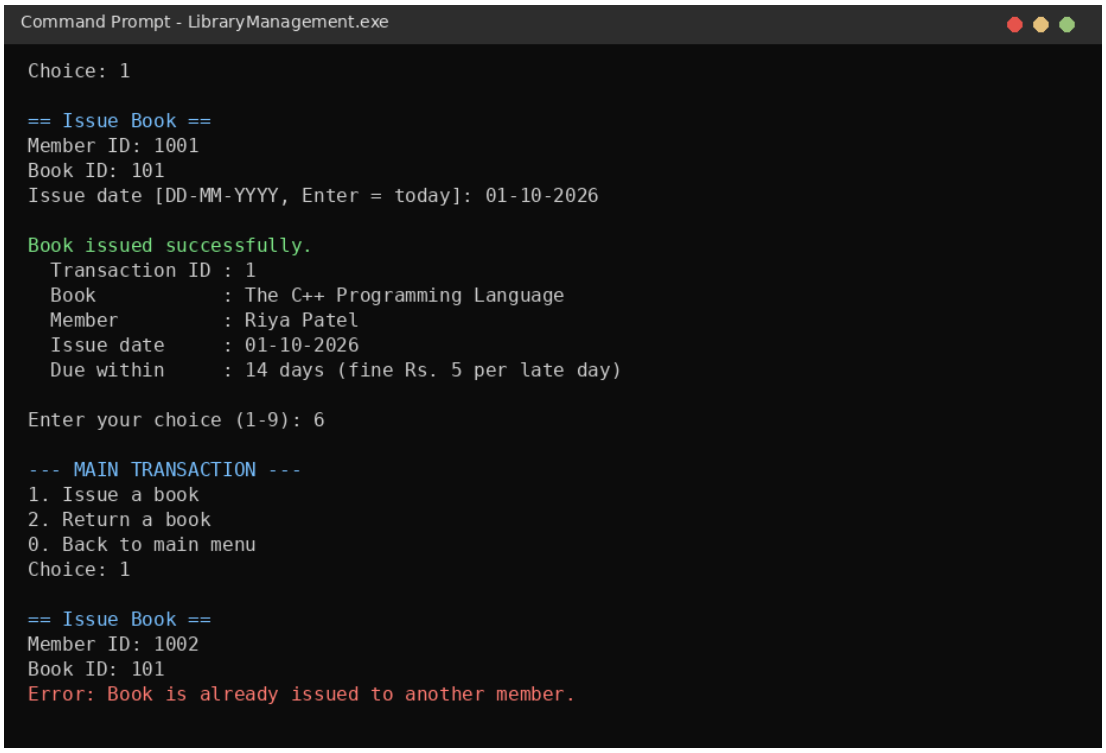


Figure 13: Issuing a book and rejecting a second issue of the same book

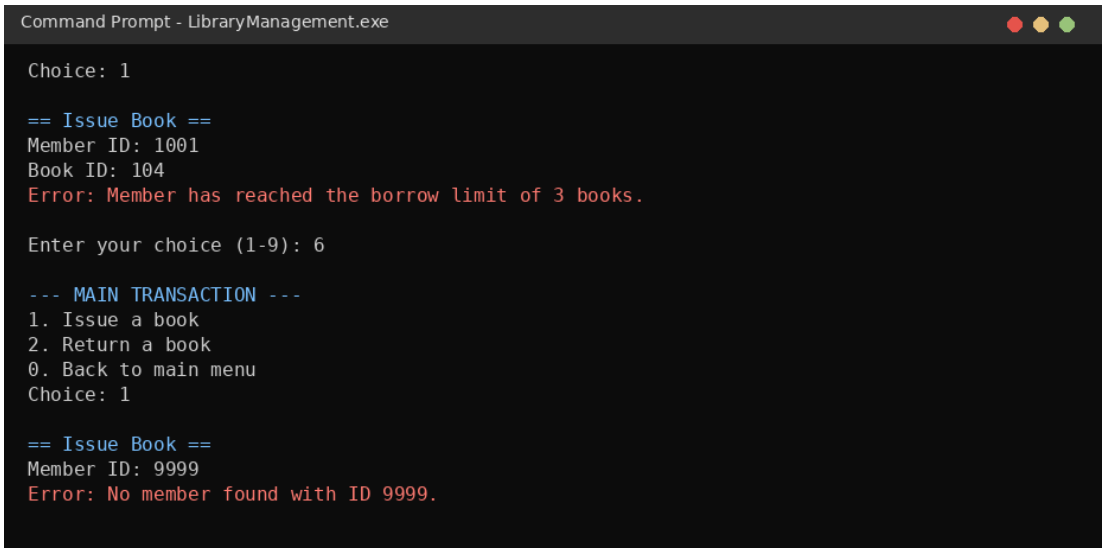


Figure 14: Borrow limit and invalid member errors

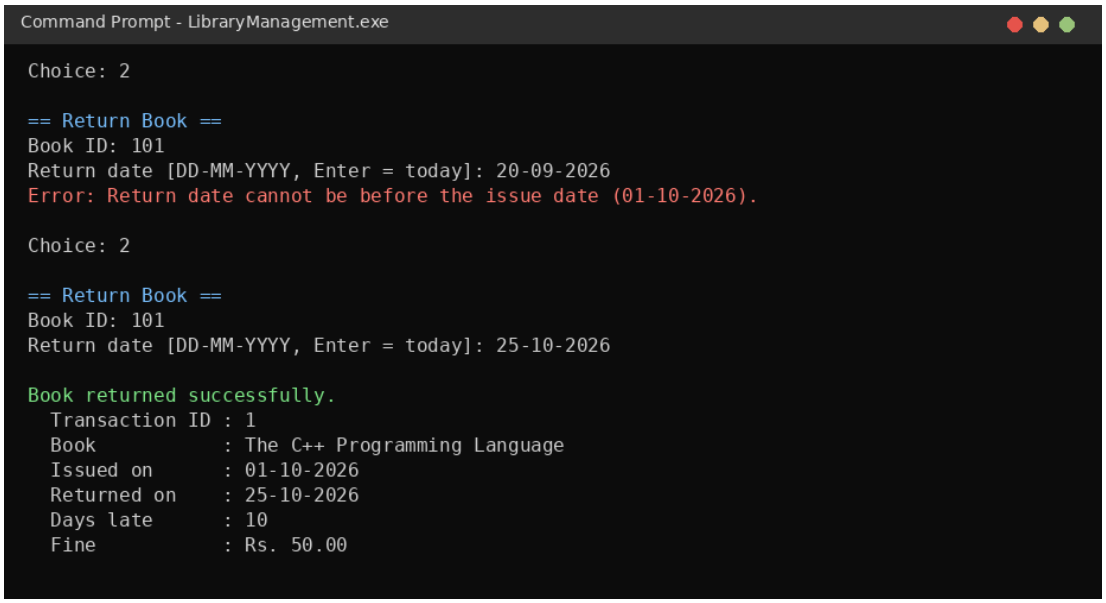


Figure 15: Returning a book with fine calculation

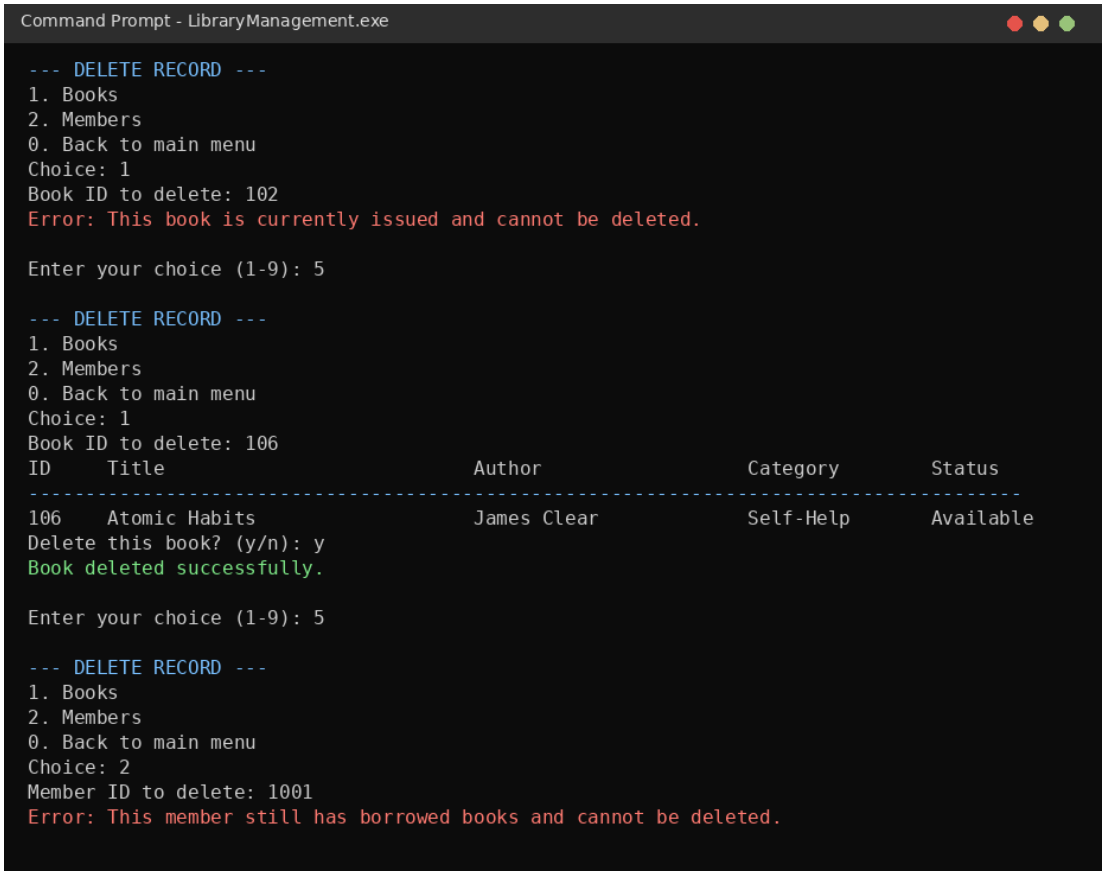


Figure 16: Delete restrictions and confirmation

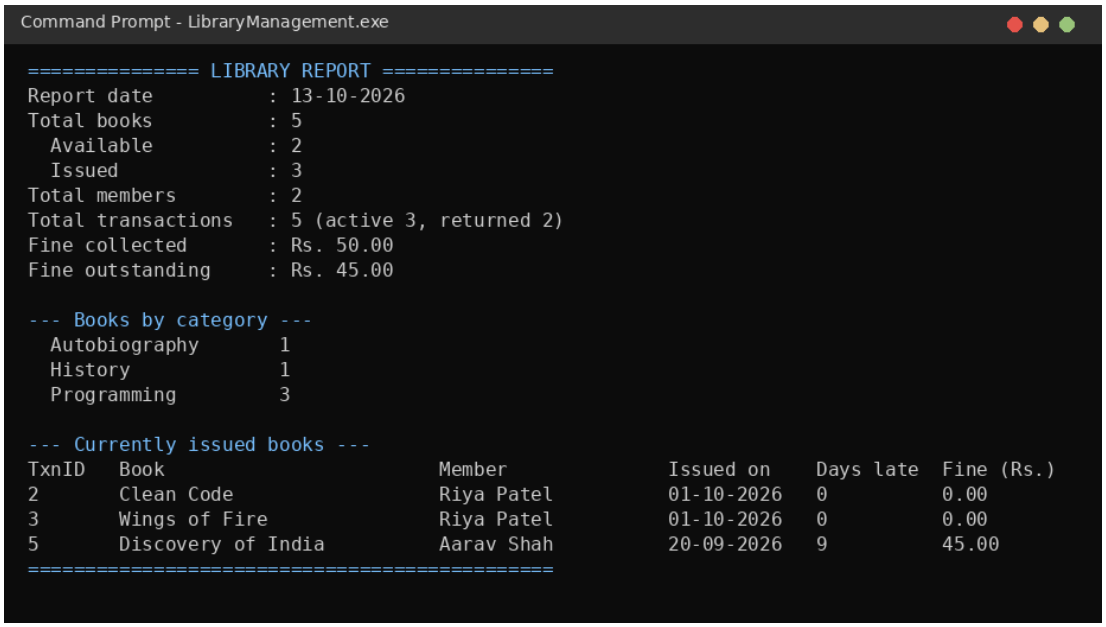


Figure 17: Report

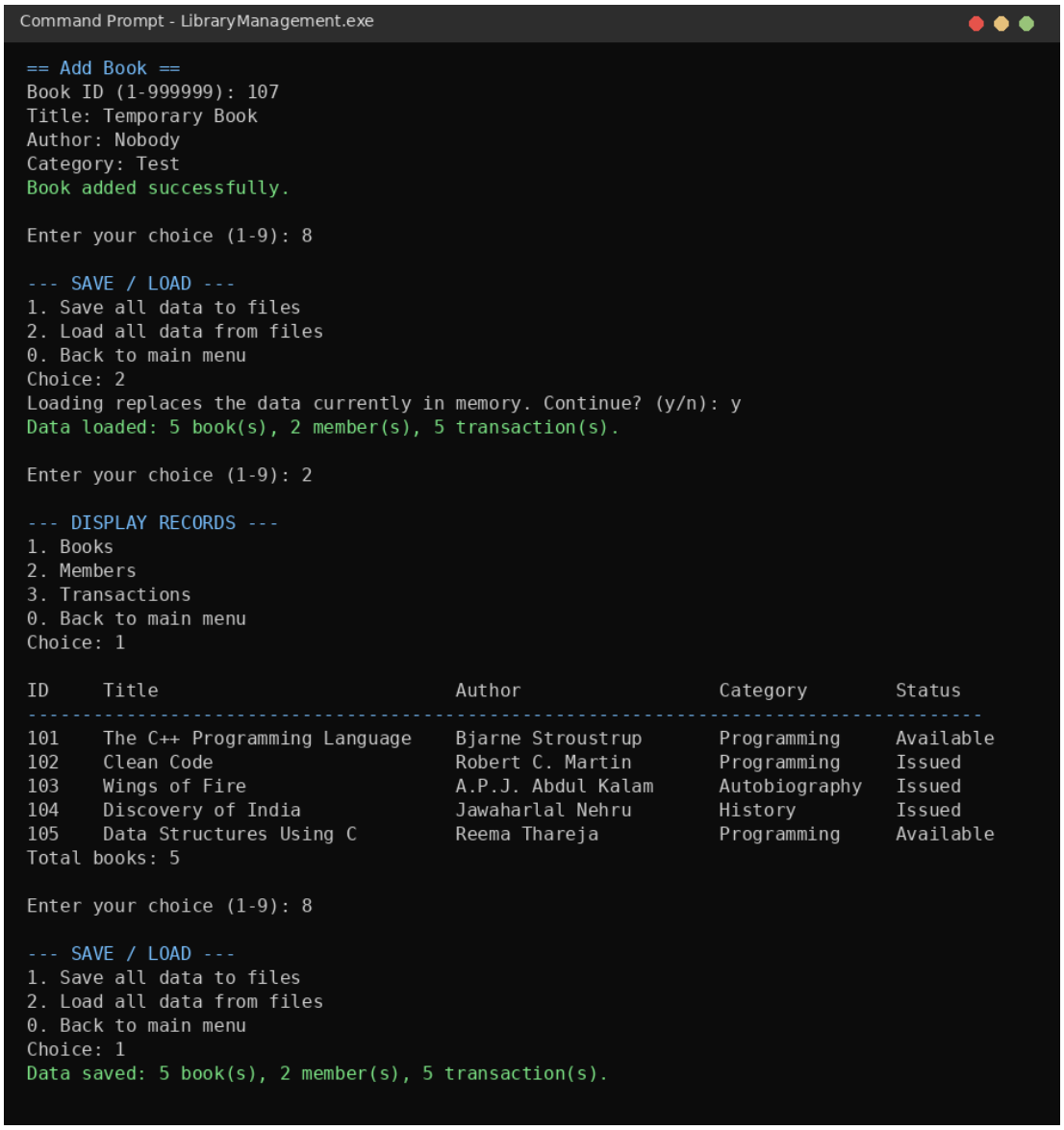


Figure 18: Save / Load persistence

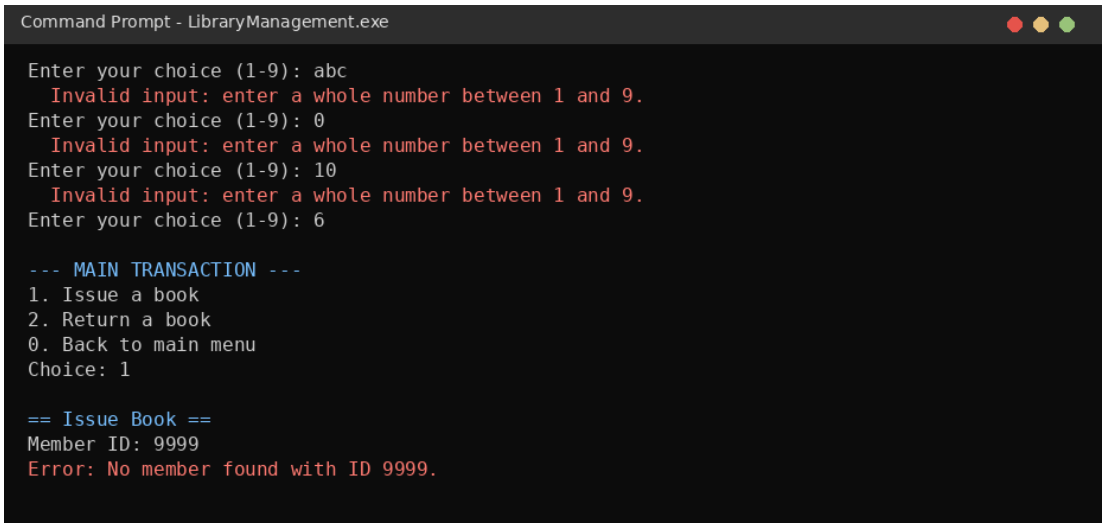


Figure 19: Input validation at the menus

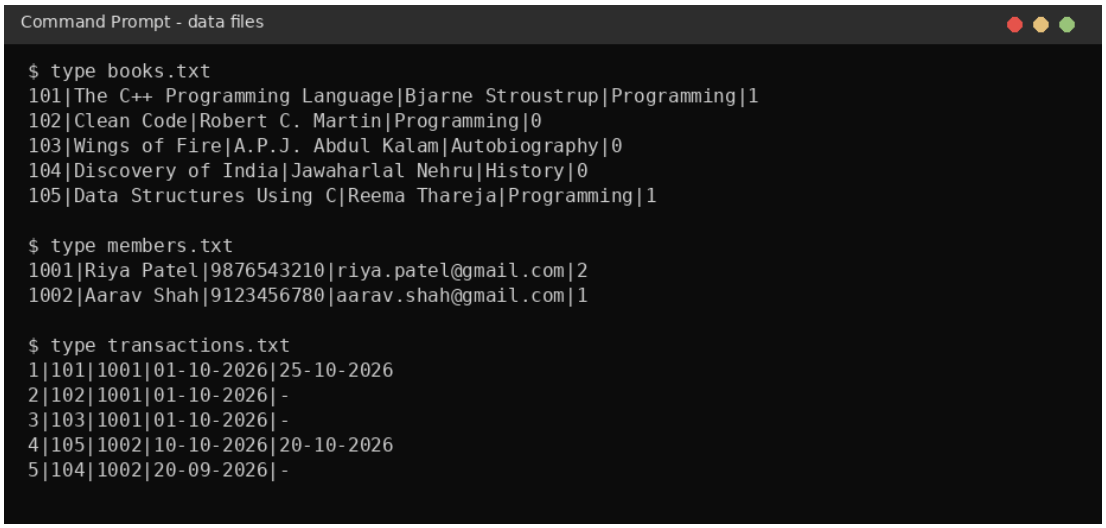


Figure 20: Data files created by the program

12. GitHub Repository

The complete source code, build files, sample data files, UML diagrams and screenshots are available in the GitHub repository:

[Paste your GitHub repository link here, e.g. https://github.com/your-username/IU2541230002-Library-Management-System]

13. Conclusion and Future Scope

The Library Management System fulfils all the requirements of the blueprint: it uses classes, encapsulation, constructors, destructors, inheritance, runtime polymorphism, dynamic memory, file handling, validation and exception handling in one working program. Testing showed that the rules for issuing, returning, fines and deletion work correctly and that data survives between runs.

Possible future improvements are a login system for librarians, reservation of issued books, a graphical user interface, storing data in a database instead of text files, and e-mail reminders for overdue books.